

Aula de hoje: esteira e destino final

01/10/2026 · Instrutor Pedro Leonardo

O que vamos fazer hoje

Na aula passada, os blocos caíam do gerador e sumiam sozinhos sem render nada. Hoje eles vão andar por uma esteira até um destino e virar dinheiro. Com isso, o dropper fica completo.

- **Esteira:** uma peça parada que empurra os blocos e o jogador.
- **Destino final:** a peça onde o bloco é vendido, some e vira dinheiro no armazém.

No fim da aula, o ciclo inteiro vai funcionar:

Gerador → Bloco → Esteira → Destino final → Armazém → Botão de coleta → Carteira

Lembrando a regra de sempre: as configurações (velocidade, tempo, valor) ficam em variáveis no topo do script. Quer mudar o jogo? Mude só essas linhas.

Etapa 1: a esteira

Por quê: o gerador solta os blocos sempre no mesmo lugar. A esteira leva cada bloco até o ponto de venda, como numa fábrica de verdade.

Como fazer:

- Embaixo do gerador, insira uma **Part** e dê o nome de **Esteira**.
- Deixe a peça comprida e baixa, por exemplo Size 4, 1, 16.
- Marque **Anchored**: a esteira não anda de verdade, ela só empurra o que está em cima.
- Escolha uma cor escura ou o material **Metal**, para parecer uma esteira.
- Se a esteira for vendida num botão, coloque-a dentro da pasta do botão, com **Transparency** em 1 e o **CanCollide** desativado, igual aos outros itens à venda.

Etapa 2: script da esteira

Por quê: sozinha, a peça é só um chão. O script dá a ela uma velocidade, e tudo que encosta em cima é empurrado nessa direção.

Como fazer: dentro de **Esteira**, insira um **Script** chamado **Esteira** e cole o código:

```
-- =====  
-- SCRIPT: ESTEIRA  
-- Empurra blocos e jogadores para a frente da peça  
-- Local: dentro da peça Esteira  
-- =====
```

```
local esteira = script.Parent
```

```
-- =====  
-- CONFIGURAÇÕES DA ESTEIRA  
-- =====
```

```
local velocidade = 8 -- quanto mais alto, mais rápido empurra (studs por segundo)

-- a esteira fica parada no lugar, mas "finge" que está andando:
-- tudo o que encosta nela recebe essa velocidade
esteira.Anchored = true

-- LookVector é a direção da frente da peça
-- multiplicar pela velocidade dá o "empurrão" naquela direção
esteira.AssemblyLinearVelocity = esteira.CFrame.LookVector * velocidade
```

Entendendo o código:

- `velocidade` é a única configuração. Quanto maior o número, mais rápido os blocos andam.
- `AssemblyLinearVelocity` é a velocidade da peça. Como ela está ancorada, não sai do lugar, mas passa essa velocidade para tudo que encosta nela.
- `LookVector` é a direção para onde a frente da peça aponta. Multiplicado pela velocidade, vira o empurrão.

O script lê a `velocidade` só uma vez, quando o jogo começa. Para testar outro valor, pare o jogo, mude o número e dê Play de novo.

Etapa 3: acertar a direção

Por quê: a esteira empurra para a frente da peça (a face `Front`). Nem sempre a frente está virada para onde você quer.

Como fazer:

- Dê **Play** e veja para onde os blocos vão.
- Se forem para o lado errado, pare o jogo e selecione a esteira.
- Aperte **Ctrl+R** para girar a peça 90 graus. Repita até a frente apontar para onde os blocos devem ir.

Girar a peça é mais fácil do que mudar o script: para fazer uma esteira em L, use duas esteiras, cada uma girada para um lado.

Etapa 4: o destino final

Por quê: o bloco precisa de um lugar para ser vendido. É no destino final que ele vira dinheiro.

Como fazer:

- No fim da esteira, insira uma **Part** e dê o nome de `DestinoFinal`.
- Marque **Anchored** e deixe a peça mais baixa que a esteira, por exemplo `Size 4, 0.5, 2`, para os blocos baterem nela.
- Pinte de verde, para o jogador entender que ali é onde o bloco vira dinheiro.
- Confira se **CanTouch** está marcado: sem ele, o destino não percebe os blocos.

Etapa 5: script do destino final

Por quê: o destino precisa fazer três coisas quando um bloco encosta: ler quanto ele vale, fazer o bloco sumir e colocar o dinheiro no armazém.

Como fazer: dentro de `DestinoFinal`, insira um **Script** chamado `DestinoFinal` e cole o código:

```
-- =====
-- SCRIPT: DESTINO FINAL
-- Quando um bloco encosta aqui, espera um pouco,
-- o bloco some e o valor vai pro armazém
-- Local: dentro da peça DestinoFinal
-- =====

-- pega o módulo Economia (o mesmo do Coletor e do painel)
local ServerScriptService = game:GetService("ServerScriptService")
local Economia = require(ServerScriptService.Economia)

local destino = script.Parent

-- =====
-- CONFIGURAÇÕES DO DESTINO
-- =====
local tempoAteSumir = 1 -- segundos entre o bloco tocar e ser destruído

-- =====
-- EVENTO: dispara toda vez que algo toca no destino
-- =====
destino.Touched:Connect(function(hit)
    -- lê a "etiqueta de preço" que o script Bloco colocou
    local valor = hit:GetAttribute("Valor")

    -- se não tem etiqueta, não é um bloco (pode ser o jogador): ignora
    if not valor then return end

    -- tira a etiqueta NA HORA, para o bloco não ser vendido 2x
    -- enquanto ele ainda está encostado esperando o tempo passar
    hit:SetAttribute("Valor", nil)

    -- espera antes de destruir
    -- (só este bloco espera; os outros continuam sendo vendidos normalmente)
    task.wait(tempoAteSumir)

    hit:Destroy() -- o bloco some
    Economia.adicionarAoArmazem(valor) -- o dinheiro vai pro armazém

    print("Bloco vendido por " .. valor .. "! Armazém: " .. Economia.armazem)
end)
```

Entendendo o código:

- `tempoAteSumir` é quanto tempo o bloco fica encostado antes de sumir. Coloque `0` para ele sumir na hora.
- `Touched` dispara toda vez que algo encosta no destino: um bloco, o pé do jogador ou qualquer outra peça.
- `GetAttribute("Valor")` lê a etiqueta de preço que o script Bloco colocou na aula passada. Só os blocos têm essa etiqueta, por isso o jogador nunca é vendido.

- `SetAttribute("Valor", nil)` tira a etiqueta logo no primeiro toque. Enquanto espera, o bloco continua encostado e o `Touched` dispara várias vezes. Sem essa linha, um bloco só seria vendido muitas vezes.
- `task.wait` faz só este bloco esperar. Os outros continuam sendo vendidos normalmente.
- `Economia.adicionarAoArmazem` é a mesma função que o gerador de teste usava. O painel e o botão de coleta continuam funcionando sem nenhuma mudança.

Se preferir que o dinheiro entre na hora do toque, e não quando o bloco some, mova a linha `Economia.adicionarAoArmazem(valor)` para antes do `task.wait`.

Testar e problemas comuns

Dê **Play**, compre o gerador e confira:

- Os blocos andam pela esteira até o destino final
- Ao subir na esteira, o jogador também é empurrado
- Os blocos somem depois de encostar no destino
- O painel do armazém sobe a cada bloco vendido
- O botão de coleta passa o dinheiro para a carteira

Problema	Causa provável	Solução
Os blocos não andam na esteira	O script não está dentro da peça Esteira	Arraste o script para dentro de Esteira
Os blocos vão para o lado errado	A frente da peça está virada para outro lado	Selecione a esteira e aperte Ctrl+R até acertar
Mudei a velocidade e nada mudou	O número foi mudado durante o Play	Pare o jogo, mude a velocidade e dê Play de novo
O bloco passa pelo destino sem sumir	O destino está baixo ou o CanTouch está desmarcado	Deixe o destino mais alto que a esteira e marque CanTouch
O bloco some, mas o armazém não sobe	O molde não tem o script Bloco, então não tem etiqueta Valor	Confira se o script Bloco está dentro do molde em Blocos
O armazém sobe mesmo sem blocos	O gerador de teste ainda está ligado	Desmarque Enabled no script Gerador antigo

Próximas aulas

- **Botão para pegar a base:** define quem é o dono de cada tycoon.
- **Armazém por tycoon:** cada base com o seu próprio dinheiro guardado.
- **Compra só para o dono:** só o dono da base pode usar os botões de compra dela.

Tarefa em aula

- **Faça toda a implementação do arquivo**

- **Monte uma linha de produção completa (gerador, esteira e destino) vendida em botões**
- **Teste velocidades e tempos diferentes até achar os que deixam o jogo mais divertido**
- **Desafio: faça uma esteira em U usando duas esteiras**